

CSA1001-SOFTWARE ENGINEERING

ASSIGNMENT 2

NAME:JASMITHA S

REG NO:192524410

Problem 1: Requirements Elicitation & Prioritization in a College Attendance System

Scenario

You are working as a Business Analyst for a college attendance management system. Students request app-based attendance checking, teachers request biometric integration, and management wants the solution to be low-cost and delivered quickly. Stakeholders have conflicting needs, and prioritization is required.

Introduction

Requirements Elicitation is the process of collecting, understanding, and documenting the needs of stakeholders before software development begins. It helps ensure that the final system satisfies business goals and user expectations.

A Business Analyst gathers requirements using techniques such as:

- Interviews
- Meetings
- Questionnaires

- Brainstorming
- Observation
- Document Analysis

Proper requirement elicitation reduces misunderstandings, avoids unnecessary development costs, and improves project success.

2. Stakeholders and Their Requirements

Stakeholder Requirement	Requirement Type
Students View attendance through a mobile application	Functional Requirement
Teachers Record attendance using biometric authentication	Functional Requirement
Management Generate attendance reports automatically	Functional Requirement
Management Complete the project within budget and timeline	Non-Functional Requirement

3. Four Well-Defined Requirements

ID	Requirement
R1	Students shall be able to view their attendance records using a mobile application at any time.

ID	Requirement
R2	Teachers shall record attendance using biometric authentication to prevent proxy attendance.
R3	The system shall automatically generate daily, weekly, and monthly attendance reports for management.
R4	The system shall be developed within the approved budget and completed before the beginning of the academic semester.

4. MoSCoW Prioritization

MoSCoW is a prioritization technique used to categorize project requirements based on their importance.

Priority	Features	Reason
Must Have	Biometric attendance, Attendance database	Essential for accurate attendance recording and system operation.
Should Have	Mobile attendance viewing, Attendance reports	Improves usability and supports academic administration.
Could Have	Attendance notifications, Graphical dashboard	Enhances user experience but is not essential for the first release.

Priority	Features	Reason
Won't Have (Current Release)	AI-based attendance prediction	Planned for future versions because of budget and time limitations.

5. Kano Model Classification

The Kano Model classifies system features based on their impact on user satisfaction.

Feature	Category	Explanation
Biometric attendance	Basic Requirement	Users expect secure and accurate attendance recording.
Mobile attendance checking	Performance Requirement	Better performance leads to greater user satisfaction.
Automatic attendance reports	Performance Requirement	Faster report generation improves management efficiency.
Attendance analytics dashboard	Excitement Requirement	Provides additional insights and delights users beyond expectations.

6. Stakeholder Conflict

Conflict

Teachers strongly recommend biometric attendance because it prevents proxy attendance and increases accuracy.

However, management wants the project to remain low-cost and be completed quickly. Purchasing biometric devices increases hardware costs and extends implementation time.

This creates a conflict between quality and budget.

7. Real-World Example

Many educational institutions such as VIT, SRM Institute of Science and Technology, Amity University, and several engineering colleges use biometric attendance systems integrated with mobile or web portals. Students can check their attendance percentage online, while faculty members generate attendance reports automatically. These systems reduce manual errors, prevent proxy attendance, and improve transparency.

8. Advantages

- Improves attendance accuracy.
- Prevents proxy attendance.
- Saves teachers' time.
- Students can monitor attendance anytime.
- Generates attendance reports automatically.

9. Disadvantages

- Biometric devices increase implementation cost.
- Hardware failures may interrupt attendance recording.
- Initial setup requires additional time.
- Biometric data raises privacy and security concerns.
- Cloud-based systems require stable internet connectivity.

10. Example

Without Prioritization

- Students request a mobile application.
- Teachers request biometric devices.
- Management requests a low-cost solution.

Result

- Development delays.
- Budget overruns.
- Stakeholder dissatisfaction.
- Frequent requirement changes.

With MoSCoW Prioritization

- Core attendance system is developed first.

- Mobile attendance application is included.
- Biometric integration is added in the next phase.
- Advanced analytics are planned for future releases.

Result

- Faster delivery.
- Lower initial cost.
- Higher stakeholder satisfaction.

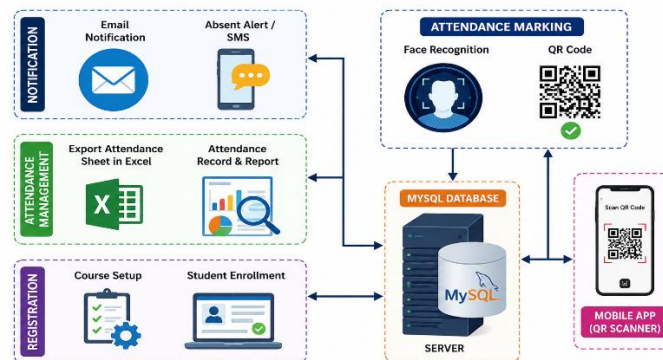


Fig. 1. System Architecture of College Attendance Management System

11. Conclusion

The College Attendance Management System demonstrates how a Business Analyst collects, analyzes, and prioritizes requirements from multiple stakeholders using Requirements Elicitation, MoSCoW Prioritization, and the Kano Model. By resolving stakeholder conflicts through phased implementation, the system achieves a balance between functionality, budget, and delivery time. This structured approach improves software quality, increases stakeholder satisfaction, minimizes project risks, and ensures the successful implementation of a modern attendance management solution.

Problem 2: Improving Collaboration with Jira & Confluence in a Hospital System

Scenario:

Your team is building a hospital appointment booking system. Developers document requirements in Jira, while designers use Confluence, leading to confusion, duplicated work, and missing updates. Teams blame each other due to miscommunication.

1. Introduction

Effective collaboration is essential in Agile software development. Jira helps teams track tasks, bugs, and project progress, while Confluence is used for documenting requirements, meeting notes, and design information. Integrating both tools ensures that all team members have access to the latest information, reducing communication gaps and improving productivity.

2. Conflict Type Identification

Conflict Type: Communication Conflict

Reason

- Developers and designers use different tools.
- Project information is not synchronized.
- Updates are missed or duplicated.
- Teams misunderstand each other's work.
- This leads to delays and blame among team members.

3. Jira + Confluence Integration

Jira and Confluence can be integrated so that project tasks and documentation remain connected.

Jira

Manages tasks and issues

Tracks progress

Assigns work

Links directly to Confluence pages

Confluence

Stores project documentation

Stores requirements and meeting notes

Shares designs and project knowledge

Links back to Jira tasks

Benefits

- Single source of project information.
- Easy access to requirements.
- Real-time updates.
- Reduced duplicate work.
- Better team collaboration.

4. Conflict Resolution Steps

Step 1: Integrate Jira and Confluence

- Link every Jira task with its corresponding Confluence document.

- Ensure all project updates are synchronized automatically.

Step 2: Improve Team Communication

- Conduct daily stand-up meetings.
- Update documents immediately after changes.
- Review requirements together before development starts.

5. Real-World Example

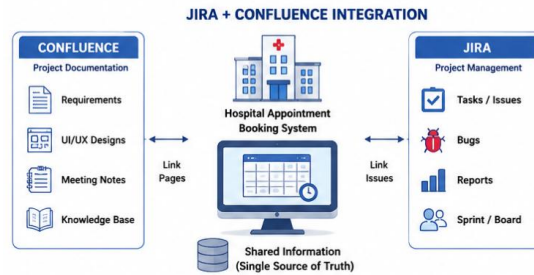
Many healthcare organizations use Jira and Confluence together to manage hospital software projects. Business Analysts write requirements in Confluence, developers track tasks in Jira, and testers update issue statuses. This integration improves communication and reduces project delays.

6. Advantages

- Improves team communication.
- Reduces duplicate work.
- Provides real-time updates.
- Centralizes project documentation.

7. Disadvantages

- Requires initial setup and training.
- Teams must regularly update documents.
- Integration may require administrator permissions.
- Overuse of documentation can increase maintenance effort.



8. Conclusion

Integrating Jira and Confluence helps development teams collaborate more effectively by connecting project tasks with documentation. It reduces communication conflicts, keeps information up to date, and improves project quality. This approach ensures successful delivery of the Hospital Appointment Booking System through better teamwork and Agile collaboration.

Problem 3: Choosing Architecture for a Scalable Food Delivery Platform

Scenario

A startup plans to develop a Food Delivery Platform similar to Swiggy or Zomato. The platform should handle thousands of orders every minute, especially during peak hours. The CTO is deciding between Monolithic, Microservices, and Serverless architectures to build a scalable and reliable system.

1. Introduction

Software architecture determines how a system is designed, developed, and maintained. For applications with a large number of users and frequent

requests, choosing the right architecture is essential to ensure high performance, scalability, reliability, and easy maintenance.

2. Recommended Architecture

Recommended Architecture: Microservices Architecture

Microservices divide the application into small, independent services. Each service performs a specific function and can be developed, deployed, and scaled separately.

3. Three Technical Reasons

Reason	Explanation
Scalability	Individual services can be scaled independently during peak hours.
Reliability	Failure of one service does not affect the entire application.
Easy Maintenance	Teams can develop, test, and deploy services independently.

4. Event-Driven Architecture

Event-Driven Architecture allows services to communicate through **events** instead of direct requests.

Benefits

- Faster communication between services.
- Supports real-time order updates.

- Improves scalability.
- Reduces system dependency.
- Better fault tolerance.

Example:

When a customer places an order, the Order Service publishes an event. The Payment Service processes the payment, and after successful payment, the Delivery Service automatically receives the event to assign a delivery partner.

5. Suitable Microservices

Microservice	Responsibility
Order Service	Creates and manages customer orders.
Payment Service	Processes online payments and refunds.
Delivery Service	Assigns delivery partners and tracks deliveries.

6. Real-World Example

Popular food delivery platforms such as Swiggy, Zomato, Uber Eats, and DoorDash use Microservices Architecture with Event-Driven Communication to manage millions of users, orders, and payments efficiently.

7. Advantages

- Highly scalable.
- Faster deployment.
- Independent service updates.

- Better fault isolation.
- Improved system performance.

8. Disadvantages

- More complex than monolithic architecture.
- Requires service monitoring.
- Increased infrastructure cost.
- More difficult debugging.

9. Conclusion

For a scalable food delivery platform, Microservices Architecture is the best choice because it supports independent scaling, high availability, and easier maintenance. Combined with Event-Driven Architecture, it enables fast communication between services, making the system reliable and capable of handling thousands of orders during peak hours.

Problem 4: API-First Development for a Hotel Booking Platform

Scenario

A software company is developing a Hotel Booking Platform. The Frontend, Backend, and Mobile teams need to work simultaneously. To avoid delays, the Project Manager decides to follow an API-First Development approach using OpenAPI (Swagger).

1. Introduction

API-First Development means designing and documenting APIs before writing the application code. This allows all development teams to work in parallel using the same API specification, improving collaboration and reducing integration issues.

2. API-First Advantage

- Enables frontend, backend, and mobile teams to work independently.
- Reduces communication and integration problems.
- Ensures consistent API documentation using Swagger/OpenAPI.
- Speeds up software development.

3. Swagger-Style API Outline

Room Availability Check API

Field	Value
Method	GET
Endpoint	/api/v1/rooms/availability
Description	Checks available rooms for selected dates.

Request Parameters

Parameter	Type
-----------	------

checkIn	Date
---------	------

checkOut	Date
----------	------

guests	Integer
--------	---------

Sample Response

```
{  
  "hotel": "ABC Hotel",  
  "availableRooms": 15,  
  "roomType": "Deluxe",  
  "price": 4500  
}
```

4. Docker Containerization Benefits

- Ensures the API runs consistently on all systems.
- Simplifies deployment and scaling across development, testing, and production environments.

5. Real-World Example

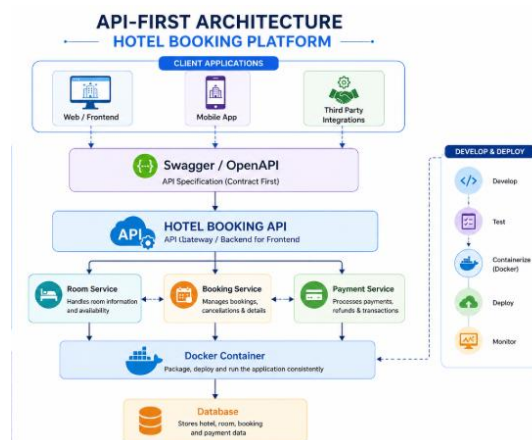
Companies such as Booking.com, Airbnb, and MakeMyTrip use API-first development and container technologies like Docker to enable multiple teams to build and deploy services efficiently.

6. Advantages

- Faster development.
- Better team collaboration.
- Easy API testing.
- Consistent deployment using Docker.
- Improved scalability.

7. Disadvantages

- Requires proper API planning.
- Initial documentation takes time.
- Version management can become complex.



8. Conclusion

API-First Development helps teams collaborate efficiently by designing APIs before implementation. Combined with Swagger documentation, API versioning, and Docker containerization, it creates a scalable, reliable.

Problem 5: Ensuring Reliability Using Formal Verification & Design by Contract

Scenario

A bank is developing a Digital Payment System where even a small software error can lead to financial losses. To ensure correctness, security, and reliability, the system must use Formal Verification and Design by Contract (DbC) principles.

1. Introduction

Formal Verification is a mathematical technique used to prove that software behaves correctly under all conditions. Design by Contract (DbC) defines rules (contracts) that every operation must follow before and after execution. These techniques are widely used in banking, aviation, healthcare, and other safety-critical systems.

2. Formal Verification Use-Case

Formal verification is mainly used in the Fund Transfer Module.

It verifies that:

- The sender has sufficient balance.
- The transfer amount is valid.
- Money is neither created nor lost.
- Every transaction is recorded correctly.
- Invalid transactions are rejected before execution.

This helps prevent financial losses and improves software correctness.

3. Design by Contract for Fund Transfer

Contract	Description
Preconditions	Sender and receiver accounts must exist, transfer amount must be greater than zero, sender must have sufficient balance.
Postconditions	Amount is deducted from sender, added to receiver, and transaction is successfully recorded.
Invariants	Total money remains unchanged, account balances never become negative, every transaction has a unique transaction ID.

4. How Contracts Improve System Reliability

- Detect errors before execution.
- Prevent invalid transactions.
- Maintain data consistency.

- Reduce software bugs.
- Increase customer trust.
- Improve system reliability.

5. Real-World Failure Example

Knight Capital Group (2012)

A software deployment error caused incorrect stock trading orders.

Result

- Around \$440 million lost in 45 minutes.
- The company suffered severe financial damage.

This failure could have been reduced through proper formal verification and Design by Contract, ensuring the software met all required conditions before deployment.

6. Real-World Applications

Formal Verification and Design by Contract are used in:

- Banking systems
- Online payment applications
- Aviation software
- Railway control systems
- Medical equipment software

7. Advantages

- Prevents financial losses.
- Detects errors early.
- Improves software quality.
- Ensures secure transactions.
- Increases customer confidence.

8. Disadvantages

- Time-consuming process.
- Requires specialized knowledge.
- Higher development cost.
- Not practical for very small applications.

9. Conclusion

Formal Verification and Design by Contract play an important role in building reliable banking software. They ensure every transaction follows predefined rules, reduce software failures, and improve system security. These techniques help banks deliver safe, accurate, and trustworthy digital payment services.